

Universidad nacional Jorge Basadre Grohmann
Facultad de ingeniería
Escuela profesional de ingeniería en informática y sistemas



Practica N°05

PRODUCTOR CONSUMIDOR

Asignatura

Sistemas Operativos

Docente

Ing. Hugo Manuel Barraza Vizcarra

Estudiante

Hugo Antonio Ordoño Quispe

2024-119049

Ciclo: 6to | **Turno:** Mañana

Tacna – Perú

2026

Análisis de resultados.

- prodcons_01.c (condición de carrera) -> Este programa implementa la solución incorrecta del problema productor-consumidor presentada en la figura 5.9 de Stallings. Explique qué problema de sincronización presenta esta implementación.

Explicación:

Al ejecutar prodcons_01.c se observa que el productor y el consumidor trabajan de manera concurrente, pero el consumidor puede intentar retirar un elemento cuando todavía no existe ninguno. Esto ocurre porque inicialmente $n = 0$ y el consumidor puede ejecutar `take()` y luego $n = n - 1$ antes de que el productor incremente n . Como resultado, durante la ejecución puede aparecer `item: 0` y posteriormente n puede llegar a -1 o incluso un número negativo más alto tal como se muestra en la figura 01. El semáforo `s` evita que ambos modifiquen n al mismo tiempo, pero no garantiza que haya un elemento disponible, por lo que se presenta la condición de carrera.

[C]	Taking	[C]	Consuming
[P]	Producing	[P]	appending
[C]	Taked	[C]	Consumed
[C]	item: 0	[P]	appended
[C]	Consuming	[P]	item: -4
[C]	Consumed	[P]	Producing
[C]	Taking	[C]	Taking
[C]	Taked	[P]	Produced
[C]	item: -1	[C]	Taked
[C]	Consuming	[C]	item: -4
[P]	Produced	[C]	Consuming
[P]	appending	[P]	appending
[C]	Consumed	[C]	Consumed
[P]	appended	[P]	appended
		[P]	item: -4

Figura 01. Inicio y fin de prodcons_01.c

- prodcons_01_buggy_01.c (error en la operación de un semáforo) -> Esta implementación introduce intencionalmente un error al sustituir una operación de espera por una operación de señalización sobre el semáforo `delay`.

Explicación

Al ejecutar prodcons_01_buggy_01.c se observa que el consumidor puede comenzar a trabajar incluso antes de que el productor haya incorporado el primer elemento. El error está en utilizar `sem_post(&delay)` al inicio del consumidor, ya que esta operación libera el semáforo en lugar de hacer que el consumidor espere. Por ello, si n todavía vale 0 , el consumidor puede ejecutar `take()` y disminuir n , provocando nuevamente valores incorrectos como -1 . Esto demuestra que cambiar `sem_wait()` por `sem_post()` altera la sincronización entre ambos hilos.

[C]	Taking	[C]	Consuming
[P]	Producing	[P]	appending
[P]	Produced	[C]	Consumed
[C]	Taked	[P]	appended
[C]	item: 0	[P]	item: -4
[C]	Consuming	[P]	Producing
[P]	appending	[C]	Taking
[C]	Consumed	[P]	Produced
[P]	appended	[C]	Taked
[P]	item: 0	[C]	item: -4
[P]	Producing	[C]	Consuming
[C]	Taking	[P]	appending
[P]	Produced	[C]	Consumed
[C]	Taked	[P]	appended
[C]	item: 0	[P]	item: -4
[C]	Consuming	[P]	Producing
[P]	appending	[C]	Taking
[P]	appended	[P]	Produced
[P]	item: 0	[C]	Taked
		[C]	item: -4

Figura 02. Inicio y fin de prodcons_01_buggy_01.c

- Identifique la instrucción incorrecta y explique cómo esta modificación afecta la sincronización entre el productor y el consumidor. ¿Qué sucede si el consumidor intenta retirar un elemento antes de que el productor haya incorporado el primero?

Explicación

La instrucción incorrecta es `sem_post(&delay)`, ya que debería utilizarse una operación de espera `sem_wait(&delay)` para controlar que el consumidor espere hasta que exista un elemento. Esta modificación hace que el consumidor pueda continuar sin que el productor haya incorporado un elemento. Si el consumidor intenta retirar un elemento cuando $n = 0$ ejecuta $n = n - 1$ haciendo que n pase a -1 , lo cual demuestra que la sincronización entre ambos hilos es incorrecta.

- prodcons_01_buggy_02.c (interbloqueo por adquisición incorrecta de semáforos) -> Esta implementación modifica el orden y la ubicación de las operaciones de sincronización del consumidor. Analice cómo estas modificaciones pueden ocasionar un interbloqueo (deadlock) entre ambos hilos.

Explicación

Al ejecutar `prodcons_01_buggy_02.c` se observa que el programa puede quedarse detenido después de mostrar algunos mensajes, tal como se muestra en la Figura 03. Esto ocurre porque el consumidor obtiene primero el semáforo `s` y luego queda esperando a `delay`, mientras que el productor necesita obtener `s` para poder incrementar `n` y realizar `sem_post(&delay)`. Como el consumidor mantiene ocupado `s`, el productor no puede continuar y, al mismo tiempo, el consumidor espera una señal del productor, produciéndose un interbloqueo o deadlock.

```
[P]      Producing
[P]      Produced
[ ]
```

Figura 03. Inicio y fin de prodcons_01_buggy_02.c

- prodcons_01_OK.c -> Este programa implementa la solución presentada en la figura 5.10 de Stallings, que incorpora una variable auxiliar local denominada m. Compare esta implementación con prodcons_01.c y explique cómo la incorporación de dicha variable permite corregir el problema identificado.

Explicación

Al ejecutar prodcons_01_OK.c se observa un comportamiento más coordinado entre el productor y el consumidor. La diferencia principal es que después de disminuir n, el consumidor guarda su valor en la variable m antes de liberar el semáforo s. Luego utiliza m para comprobar si el buffer quedó vacío. De esta manera, la comprobación no depende de que otro hilo modifique n después de liberar s, corrigiendo el problema de sincronización presente en prodcons_01.c y permitiendo que los hilos continúen ejecutándose correctamente.

```
[P]      Producing
[P]      Produced
[P]      appending
[P]      appended
[P]      item: 1
[P]      Producing
[C]      Taking
[P]      Produced
[C]      Taked
[C]      item: 1
[C]      Consuming
[P]      appending
[C]      Consumed
[P]      appended
[P]      item: 1
[P]      Producing
[C]      Taking
[P]      Produced
[C]      Taked
[C]      item: 1
```

Figura 04. Ejecución de prodcons_01_OK.c

- Ejercicio 3.

```
[P0] Producing 0 ...  
[P0] Producing 1 ...  
[P1] Producing 0 ...  
[P1] Producing 1 ...  
--> [C0] consumed 0  
--> [C0] consumed 1  
[P2] Producing 0 ...  
[P2] Producing 1 ...  
--> [C1] consumed 0  
--> [C1] consumed 1  
--> [C2] consumed 0  
--> [C2] consumed 1  
[P2] Producing 2 ...  
[P2] Producing 3 ...  
--> [C0] consumed 2  
--> [C0] consumed 3  
[P0] Producing 2 ...  
[P0] Producing 3 ...  
--> [C2] consumed 2  
--> [C2] consumed 3  
[P1] Producing 2 ...  
[P1] Producing 3 ...  
--> [C1] consumed 2  
--> [C1] consumed 3
```

Figura 05. Ejecución del ejercicio3.c